



RUNBOOK · DRAFT FOR REVIEW

VIRTUAL CONTROL

VMWARE CLOUD FOUNDATION SOLUTIONS

NSX Manager Redeploy & Transport-Node Rebuild

Clean recovery from the post-vCenter-redeploy VDS-UUID / orphaned-realization failure (8804 / 8012 / 90325).

v1.6

VCF 9.0.1 · NSX 9.0.1

© 2026 Virtual Control LLC — All Rights Reserved

INTERNAL

NSX Manager Redeploy & Transport-Node Rebuild

Prepared by: Virtual Control LLC **Date:** June 10, 2026 **Document Version:** 1.6 — **DRAFT (review before executing)** **Classification:** Internal

This runbook destroys and rebuilds the NSX management plane. It is the agreed fallback after in-place reconcile was **proven exhausted** (June 9 deep-dive + June 10 live re-test: force-delete of the orphaned transport-node state returned HTTP 200 but did **not** clear the zombie realization; retry counters frozen since June 9; no TN config / no transport-node-collection behind them; the compute manager cannot clear NSX ownership on a VDS object that no longer exists — 8012 → 90325). **Do not execute any step until the two decisions in §0.3 are made and a change window is open.**

- NSX Manager Redeploy & Transport-Node Rebuild
 - 0. Purpose, basis & decisions
 - 0.1 The failure (verified live, 2026-06-10)
 - 0.2 Why redeploy (and why NOT restore from backup)
 - 0.3 TWO decisions to make before executing
 - 0.4 Safety
 - 1. Pre-flight — capture exact config & confirm safe (READ-ONLY)
 - 2. Phase A — Decommission the broken NSX Manager & clean the hosts
 - A1 — Power off the old NSX Manager VM & free its name
 - A2 — (FALLBACK ONLY) Clean residual NSX from a host (NOT officially supported — KB 312644 / 322412)
 - 3. Phase B — Deploy a fresh NSX Manager (Broadcom: "Deploy NSX Manager"; lab method: PowerCLI Import-VApp)
 - New to PowerShell? Read this first — what to copy/paste, when, and why
 - 4. Phase C — Configure NSX (verified POST bodies from your NSX9-Transport-Node-Profile-Schema-Reference)
 - C1 — Register the compute manager
 - C2 — Transport Zones (policy API)
 - C3 — Uplink profile (manager API) — teaming LOADBALANCE_SRCID, no MTU
 - C4 — TEP IP pool (policy API, two-step)
 - 5. Phase D — Transport Node Profile + bind to the cluster (manager API)
 - D1 — Re-fetch the current manager-API UUIDs (do not reuse the schema doc's)
 - D2 — Create the TNP (POST — manager API; the policy API path returns 405)
 - D3 — Bind the TNP to the cluster (TransportNodeCollection)
 - 6. Phase E — Verify
 - 7. Phase F — Re-establish SDDC Manager & VCF Operations
 - 8. Rollback / fallback
 - 9. Captured configuration (fill from §1 before executing)

- 10. Document Information
 - Revision History

0. Purpose, basis & decisions

0.1 The failure (verified live, 2026-06-10)

- 4 host transport nodes stuck `pending` / **8804**: *"Cannot find the HostSwitch `50 37 13 3f 96 e4 3e 93-1d f0 3b 01 d4 76 6a f8` of type VDS."*
- Compute manager `REGISTERED_WITH_ERRORS` / **90325** → **8012**: *"Unable to clear NSX ownership from MO `.../50 37 13 3f...` — object could not be found."*
- Cause: the vCenter **redeploy** recreated `vcenter-cl01-vds01` with a **new UUID** `50 12 35 19 fa f5 be 0a-71 8d ab e0 b5 e6 b9 cf`; NSX's stored host-switch/ownership references point at the **old, deleted** UUID. NSX Manager itself is healthy (single-node, STABLE); vSAN healthy; hosts `Connected` (NSX-RPC) .

0.2 Why redeploy (and why NOT restore from backup)

- In-place clear is impossible: NSX is trying to clear ownership on an object that is gone, and the realization state is orphaned below the config layer (nothing API-addressable). Confirmed twice.
- **Do NOT restore the June-9 NSX backup onto the new appliance.** The backup post-dates the vCenter redeploy and carries the **same stale host-switch/ownership references** — restoring it would reintroduce this exact failure. **Rebuild configuration fresh** (it is small — see §1). The backup is kept only as an emergency reference.

0.3 TWO decisions to make before executing

1. **Who owns NSX host prep — SDDC Manager or NSX-native?** The cluster is `lifecycleManaged: true` . If this NSX is meant to be **SDDC-Manager-managed**, the sanctioned rebuild is via SDDC Manager (Add/Re-deploy NSX), and SDDC Manager's NSX inventory must be reconciled first. In this lab NSX was deployed semi-manually (PowerCLI `Import-VApp`), and SDDC Manager's NSX inventory is currently empty — so the **NSX-native rebuild below is the practical path**, with SDDC Manager re-pointed afterward (§F). **Confirm which model applies before starting.**
2. **Fresh deploy (recommended) vs. deploy+restore (rejected here).** Per §0.2, this runbook assumes **fresh deploy + reconfigure**.

0.4 Safety

- **0 segments / 0 edges** confirmed — no overlay workload to lose. **Verify again at §1.**
- Per the fragile-single-host rules: **disable cluster HA** during the host-cleanup/re-prep window (re-prep touches host networking); work **one host at a time** in maintenance mode; re-check **vSAN object health** between hosts.

- Passwords are <vault> references throughout.

1. Pre-flight — capture exact config & confirm safe (READ-ONLY)

Run these **before** decommissioning so the rebuild is exact. (NSX=192.168.1.70 , -u 'admin:<vault>' .)

No-guessing rule for this runbook. Every value tagged [confirm from capture] / [paste] (Transport VLAN, TEP pool CIDR/range/gateway, uplink teaming policy, OVF property keys, the exact management portgroup name, and the appliance form-factor/size) is **not yet verified** and **must be read from the live system in this section** — never assumed. If a capture command returns something different from the "values already on record" table below, **the live value wins**. Do not proceed past Phase A with any [confirm] field still blank.

```
# Run in Git Bash or WSL (NOT Windows cmd). Each should show "result_count": 0
curl -sk -u 'admin:<vault>' "https://$NSX/policy/api/v1/infra/segments"
curl -sk -u 'admin:<vault>' "https://$NSX/api/v1/logical-ports"
curl -sk -u 'admin:<vault>' "https://$NSX/api/v1/edge-clusters"

# Capture exact values to recreate (read the JSON, record into Section 9):
# TZ names + types:
curl -sk -u 'admin:<vault>' "https://$NSX/api/v1/transport-zones"
# uplink profile (teaming policy + Transport VLAN):
curl -sk -u 'admin:<vault>' "https://$NSX/api/v1/host-switch-profiles"
# TEP pool (CIDR / range / gateway):
curl -sk -u 'admin:<vault>' "https://$NSX/api/v1/pools/ip-pools"
# TNP (VDS, uplinks, TZ bindings):
curl -sk -u 'admin:<vault>' "https://$NSX/api/v1/transport-node-profiles"
```

Verified values already on record (confirm unchanged):

Item	Value
NSX Manager node / VIP	nsx-manager.lab.local 192.168.1.71 / VIP 192.168.1.70 (single node)
Compute manager	vCenter 192.168.1.69 , administrator@vsphere.local
Target VDS (LIVE)	vcenter-cl01-vds01, UUID 50 12 35 19 fa f5 be 0a-71 8d ab e0 b5 e6 b9 cf , MTU 1700
VDS uplinks	dvUplink1, dvUplink2 → uplink-1 / uplink-2 (is_migrate_pnics: false)
Overlay TZ	vcenter-cl01-tz-overlay01
VLAN TZ	vcenter-cl01-tz-vlan01
Uplink profile	vcenter-cl01-up-host01 ; teaming LOADBALANCE_SRCID (uplink-1/uplink-2 active, no standby); Transport VLAN = 0 (untagged); no MTU (As-Built + TNP-Schema)

Item	Value
TEP IP pool	vcenter-cl01-tep-pool-01 ; subnet 192.168.13.0/24 , range .101-.120 (20), gateway 192.168.13.1 (As-Built + TNP-Schema)
Transport Node Profile	vcenter-cl01-tnp (rebuild)
Cluster	vcenter-cl01 (domain-cl0) — hosts esxi01-04 = 192.168.1.74 / .75 / .76 / .82
NSX OVA	nsx-unified-appliance-9.0.1.0.24952111.ova (verified on disk, 11.23 GB). A same-size duplicate ... 24952114.ova exists — identical content; either deploys the same Manager . Depot C:\VCF-Depot\PROD\COMP\NSX_T_MANAGER\ / .160 . (NOT the nsx-edge OVA — 0 edges in use.)
PowerCLI	VMware.PowerCLI 13.3.0 installed; Connect-VIServer / Get-OvfConfiguration / Import-VApp / Get-VM / Start-VM all present (verified 2026-06-11).

Action: copy the captured TZ / uplink-profile / TEP-pool / TNP JSON into §9 of this doc before proceeding.

2. Phase A — Decommission the broken NSX Manager & clean the hosts

A1 — Power off the old NSX Manager VM & free its name

- Old appliance = nsx-manager , vmid 17 on esxi01 (.74), on vcenter-cl01-ds-vsan01 .
- **Two different "names" — keep them straight:** the **hostname / FQDN / IP** (nsx-manager.lab.local / .71) is set *inside* the appliance via OVF and is what NSX/DNS/certs use — **powering the old VM off frees it** so the new one can reuse it. The **vCenter VM display name** (nsx-manager) is just a label; two VMs can't share it in one folder, so rename the old one.

Step 1 — graceful power-off (frees the IP + hostname):

```
# esxi01 (192.168.1.74) — use power.off 17 if it won't stop gracefully
vim-cmd vmsvc/power.shutdown 17
```

Step 2 — rename the old VM so Phase B's -Name 'nsx-manager' is free (keep it as rollback, do NOT delete yet):

```
# Workstation PowerCLI
Connect-VIServer 192.168.1.69 -User administrator@vsphere.local -Password '<vault>'
Get-VM nsx-manager | Set-VM -Name 'nsx-manager-OLD-2026-06-10' -Confirm:$false
```

- Rename changes only the vCenter label — it does **not** move files (vSAN folders are UUID-named, so no collision). Keep nsx-manager-OLD-... **powered off** as your rollback; **delete it only after §E verifies** the new manager.
- *Alternative:* power off → **Remove from Inventory** (also frees the name; leaves the files to re-register if needed).

A2 — (FALLBACK ONLY) Clean residual NSX from a host (*NOT officially supported* — KB 312644 / 322412)

Do this ONLY if a specific host fails to re-prep in Phase D3 — do NOT pre-clean. Per your prior rebuild (schema-doc **L-8**), hosts that still carry the NSX VIBs normally transition `Not Configured` → `success` in **3–5 min** once the TNP/TNC is applied. So **run Phase B→D first and leave the hosts alone**; only use this manual removal if a host is still not `success` after ~10 min (`/api/v1/transport-nodes/<id>/state` shows a `failure_message`). It is **not VMware-supported** — last resort, **one host at a time**, in maintenance mode (`ensureObjectAccessibility`), with **cluster HA disabled**.

Per host (esxi01-04), as root via SSH:

```
# 1) Enter maintenance mode (exact value is ensureObjectAccessibility – case-sensitive; we are NOT
changing vSAN disks)
esxcli system maintenanceMode set -e true -m ensureObjectAccessibility
# 2) Clear DFW filters, then remove NSX
vsipioctl clearallfilters -Override 2>/dev/null
nsxcli -c "del nsx" # supported cleanup entry point; if it errors "Busy", see below
# 3) If "del nsx" fails (KB 322412 Busy on com.vmware.nsx.vd12), remove VIBs manually then reboot:
#   esxcli software vib list | grep -i nsx # enumerate
#   esxcli software vib remove -n nsx-esx-datapath -n nsx-mpa -n nsx-proxy -n nsx-platform-client ... #
all nsx-*
# 4) Reboot the host, exit maintenance mode, confirm clean:
reboot
# after boot:
esxcli software vib list | grep -i nsx # expect: none
esxcli system maintenanceMode set -e false
```

- **Between hosts:** `esxcli vsan debug object health summary get` → all `0` before moving to the next host.

3. Phase B — Deploy a fresh NSX Manager (*Broadcom: "Deploy NSX Manager"; lab method: PowerCLI Import-VApp*)

Lab note: **two deploy methods fail on this OVA** — the signed `.ova` makes `Import-VApp` **hard-fail on its certificate** ("not valid (Untrusted)"), and the point-and-click **UI deploy silently drops the OVF customization** (2026-06-11 — the appliance came up on DHCP `.13` with no config). The reliable path is **B0: extract the OVA, delete its `.cert`, and `Import-VApp` the unsigned `.ovf`** (which preserves the `vAppConfig` — detail in *NSX9-OVA-PowerCLI-ImportVApp-vs-ovftool*), then **B4: verify the settings applied before powering on. Run these in PowerShell, NOT Git Bash.** = uses a single space so it copies clean.

New to PowerShell? Read this first — what to copy/paste, when, and why

What you're using. Two things working together: **PowerShell** (a Windows text-command window — a way to tell the computer to do things by typing instead of clicking) and **PowerCLI** (a free VMware add-on that teaches PowerShell how to talk to vCenter). Both are already installed. *This is a different window from "Git Bash," which you'll use later in Phase C — for Phase B, use PowerShell (Start menu → "PowerShell" → the blue icon).*

Why type commands instead of using the deploy wizard? The point-and-click "Deploy OVF Template" wizard has, in this lab, **silently dropped the NSX settings** (the 2026-05-06 lesson) — the appliance came up with no network config. The command method below sets **every** value reliably, so we do it once, correctly.

The big idea — five short blocks, run in order:

- **Block B0 = "unwrap the box."** The signed `.ova` trips a certificate error, so we extract it and delete the signature file, leaving a plain `.ovf` we can deploy. *(Like taking the item out of its shrink-wrap.)*
- **Block B1 = "look things up."** Two details aren't known until we ask vCenter: the **exact host name** and the **management network name**. B1 connects and prints them. *(Like checking the address and the menu before you order.)*
- **Block B2 = "fill out the form."** This types every NSX setting (name, IP, gateway, DNS, passwords, network) into a configuration object called `$cfg`. **Nothing is deployed yet.** *(Like typing into an online order form.)*
- **Block B3 = "press Go."** This builds the appliance — **but does NOT power it on yet.** *(Like clicking "Place Order.")*
- **Block B4 = "check the receipt, then turn it on."** Before powering on, we confirm the settings actually stuck — the step that catches the 2026-06-11 failure. Only then do we start it. *(Like checking your order is right before you drive off.)*

We split it up **so you can double-check before anything is created or powered on** — far safer than one giant paste.

How to actually copy/paste: 1. Highlight a whole block (all its lines) and press **Ctrl+C**. 2. Click into the blue PowerShell window. 3. Paste with **right-click** (PowerShell's paste) or **Ctrl+V**. 4. Press **Enter** to run the last line. PowerShell runs the lines top-to-bottom.

Replacing <...> placeholders. Where you see angle brackets like `<vCenter SSO pw>`, replace the **whole thing including the < >** with the real value in quotes — e.g. `'MyActualPassword!'`. **Leaving the brackets in is the #1 mistake** (it sends the literal `< >` and fails).

"Nothing happened" usually means it worked. When B2 sets a value, PowerShell stays **silent** if it succeeded — *silence = success*. A real problem shows up as **red text**. So **no red = good**.

Always sanity-check before B3. Before deploying, we ask PowerShell to *read back* a few key values (network, IP, hostname). If they print what you expect, the form is filled correctly.

What the deploy looks like (B3). After you hit Enter, the cursor just **sits there for several minutes with no progress bar** while it copies the 11 GB appliance — **that is normal, NOT frozen; do not cancel it**. When it finishes it prints the VM and powers it on. NSX then **boots for ~15–30 minutes** before its web page (`https://192.168.1.71/`) answers.

B0 — extract the OVA and remove its signature (this is what makes `Import-VApp` work — the signed `.ova` fails with a certificate error):

```
$ova = 'I:\VCF-Depot\PROD\COMP\NSX_T_MANAGER\nsx-unified-appliance-9.0.1.0.24952111.ova'
$dir = 'C:\VCF-Depot\nsx-extract'
New-Item -ItemType Directory -Force -Path $dir | Out-Null
tar -xf $ova -C $dir
Remove-Item "$dir\*.cert" -Force
$ovf = (Get-ChildItem "$dir\*.ovf").FullName
$ovf
```

The last line prints the extracted descriptor (e.g. `...24952114.ovf`). With the `.cert` deleted there is no signature for PowerCLI to reject, and the `.ovf` keeps **every** OVF property — unlike the UI deploy, which silently dropped them (2026-06-11). (*Verified-reliable for the NSX 9 OVA; detail in NSX9-OVA-PowerCLI-ImportVApp-vs-ovftool.*)

B1 — connect, load config, discover the two unknowns:

```
Set-PowerCLIConfiguration -InvalidCertificateAction Ignore -Confirm:$false
Connect-VIServer 192.168.1.69 -User administrator@vsphere.local -Password '<vCenter SSO pw>'
$vmh = Get-VMHost esxi01.lab.local # vCenter inventory name (FQDN), NOT the IP
$ds = Get-Datastore vcenter-cl01-ds-vsan01
$cfg = Get-OvfConfiguration $ovf
Get-VirtualPortGroup -VMHost $vmh | Select-Object Name
$cfg.NetworkMapping
```

The last two lines show the **mgmt portgroup name** and the OVA's **network label** — note both.

B2 — set the values (edit the first two lines to your portgroup + chosen NSX password):

```
$pg = 'pg-vm-mgmt'
$nsxpw = '<NSX root/admin/audit password - 12+ chars, complexity>'
$cfg.DeploymentOption.Value = 'small'
foreach ($n in $cfg.NetworkMapping.PSObject.Properties.Name) { $cfg.NetworkMapping.$n.Value = $pg }
$cfg.Common.nsx_hostname.Value = 'nsx-manager.lab.local'
$cfg.Common.nsx_role.Value = 'NSX Manager'
$cfg.Common.nsx_ip_0.Value = '192.168.1.71'
$cfg.Common.nsx_netmask_0.Value = '255.255.255.0'
$cfg.Common.nsx_gateway_0.Value = '192.168.1.1'
$cfg.Common.nsx_dns1_0.Value = '192.168.1.230'
$cfg.Common.nsx_domain_0.Value = 'lab.local'
$cfg.Common.nsx_ntp_0.Value = '192.168.1.230'
$cfg.Common.nsx_passwd_0.Value = $nsxpw
$cfg.Common.nsx_cli_username.Value = 'admin'
$cfg.Common.nsx_cli_passwd_0.Value = $nsxpw
$cfg.Common.nsx_cli_audit_username.Value = 'audit'
$cfg.Common.nsx_cli_audit_passwd_0.Value = $nsxpw
$cfg.Common.nsx_isSSHEnabled.Value = $true
$cfg.Common.nsx_allowSSHRootLogin.Value = $true
```

`DeploymentOption = small` (4 vCPU / 16 GB — supported for lab/PoC, fully functional for TZ/uplink/TEP/TNP/host-prep, saves 8 GB RAM vs `medium`). Leave blank (fresh single node): `mpIp` / `mpToken` / `mpThumbprint` / `mpNodeId` / `mpClusterId`, all IPv6 (`nsx_ip6_0` / `nsx_netmask6_0` / `nsx_gateway6_0`), and `nsx_grub_passwd`.

Sanity-check before deploying — read the values back (expect `192.168.1.71`, the FQDN, `small`, your portgroup):


```
$cfg.Common.nsx_ip_0.Value; $cfg.Common.nsx_hostname.Value; $cfg.DeploymentOption.Value  
$cfg.NetworkMapping.PSObject.Properties | ForEach-Object { $_.Value.Value }
```

B3 — deploy from the extracted .ovf (DO NOT power on yet). First **delete the old/broken nsx-manager VM** (Delete from Disk) so the name + vSAN folder are free:

```
Import-VApp -Source $ovf -OvfConfiguration $cfg -VMHost $vmh -Datastore $ds -Name 'nsx-manager' -  
DiskStorageFormat Thin
```

The import sits for several minutes with no progress bar while it copies — **normal, do not cancel**.

B4 — VERIFY the settings actually applied, THEN power on (*the gate that catches the 2026-06-11 silent-drop*):

```
(Get-VM nsx-manager).ExtensionData.Config.VAppConfig.Property | Select-Object Id, Value
```

You **must** see `nsx_ip_0 = 192.168.1.71` and `nsx_hostname = nsx-manager.lab.local` (plus the rest). - **Present** → **power on**: `Start-VM nsx-manager` - **Empty / missing** → **STOP, do NOT power on**. The config was dropped — recheck B0 (was the `.cert` removed? is `$ovf` the extracted file, not the `.ova` ?) and that `-OvfConfiguration $cfg` was passed in B3. (*Host-side confirmation: the VM's .vmx should now contain guestinfo / nsx_ip_0 entries — empty .vmx = dropped config.*) - After boot, confirm UI at `https://192.168.1.71/` and **set the cluster VIP 192.168.1.70** (System → Appliances → Virtual IP) so the original VIP/cert SANs are reused. - **License**: NSX runs in **evaluation mode (~90 days)** out of the box — **no license needed to rebuild**. Apply the license later (System → Licenses) when it arrives via (premier-partner channel).

4. Phase C — Configure NSX (verified POST bodies from your *NSX9-Transport-Node-Profile-Schema-Reference*)

Run in Git Bash / WSL. Set once (use the new node `.71` until you set the VIP `.70` in Phase B):

```
NSX=https://192.168.1.71  
P='admin:<vault>'
```

No-guessing — these IDs are PER-REBUILD; re-fetch them live, do NOT copy the schema doc's UUIDs (its `3fbf2d31...`, `ce4eb145...`, `0b6b1dc5...`, `15e0debc...`, and `50 37 13 3f...` are from the 2026-05-06 session and are now stale): - **VDS host_switch_id** = the **current** live value `50 12 35 19 fa f5 be 0a-71 8d ab e0 b5 e6 b9 cf` — re-confirm with `esxcli network vswitch dvs vmware list` on any host. - **TZ / IP-pool / uplink-profile UUIDs** = re-fetch after you create each (manager-API UUIDs, **not** policy display names — that was the #1 failure mode, errors 520033 / 9543). - **compute_collection_id** CM-UUID prefix = re-fetch after C1 (it changes on every CM re-registration).

C1 — Register the compute manager

UI: **System** → **Fabric** → **Compute Managers** → **Add** → 192.168.1.69, administrator@vsphere.local / <vault>, **Enable Trust** = **Yes**, accept thumbprint → wait **Registered / Up, no errors**. (A fresh NSX has no stale ownership, so it registers clean.)

C2 — Transport Zones (policy API)

```
curl -sk -u "$P" -X PATCH "$NSX/policy/api/v1/infra/sites/default/enforcement-points/default/transport-zones/vcenter-cl01-tz-overlay01" -H "Content-Type: application/json" -d '{"display_name":"vcenter-cl01-tz-overlay01","tz_type":"OVERLAY_STANDARD"}'
curl -sk -u "$P" -X PATCH "$NSX/policy/api/v1/infra/sites/default/enforcement-points/default/transport-zones/vcenter-cl01-tz-vlan01" -H "Content-Type: application/json" -d '{"display_name":"vcenter-cl01-tz-vlan01","tz_type":"VLAN_BACKED"}'
```

C3 — Uplink profile (manager API) — teaming `LOADBALANCE_SRCID`, **no MTU**

```
curl -sk -u "$P" -X POST "$NSX/api/v1/host-switch-profiles" -H "Content-Type: application/json" -d '{"display_name":"vcenter-cl01-up-host01","resource_type":"UplinkHostSwitchProfile","lags":[],"teaming":{"policy":"LOADBALANCE_SRCID","active_list":[{"uplink_name":"uplink-1","uplink_type":"PNIC"}, {"uplink_name":"uplink-2","uplink_type":"PNIC"}],"standby_list":[],"transport_vlan":0,"named_teamings":[]}]'
```

Do **not** set MTU here — the VDS owns it (MTU in the uplink profile = Error 9531). Record the returned `id`.

C4 — TEP IP pool (policy API, two-step)

```
curl -sk -u "$P" -X PATCH "$NSX/policy/api/v1/infra/ip-pools/vcenter-cl01-tep-pool-01" -H "Content-Type: application/json" -d '{"display_name":"vcenter-cl01-tep-pool-01","resource_type":"IpAddressPool"}'
curl -sk -u "$P" -X PATCH "$NSX/policy/api/v1/infra/ip-pools/vcenter-cl01-tep-pool-01/ip-subnets/tep-subnet" -H "Content-Type: application/json" -d '{"resource_type":"IpAddressPoolStaticSubnet","display_name":"tep-subnet","cidr":"192.168.13.0/24","gateway_ip":"192.168.13.1","allocation_ranges":[{"start":"192.168.13.101","end":"192.168.13.120"}]}'
```

5. Phase D — Transport Node Profile + bind to the cluster (*manager API*)

D1 — Re-fetch the current manager-API UUIDs (do not reuse the schema doc's)

```
curl -sk -u "$P" "$NSX/api/v1/transport-zones" | python -m json.tool | grep -E '"display_name"|"id"'
curl -sk -u "$P" "$NSX/api/v1/pools/ip-pools" | python -m json.tool | grep -E '"display_name"|"id"'
```

Current VDS UUID — run on any host: `esxcli network vswitch dvs vmware list` → **VDS ID** = 50 12 35 19 fa f5 be 0a-71 8d ab e0 b5 e6 b9 cf.

D2 — Create the TNP (POST — manager API; the policy API path returns 405)

Fill the four `<...>` from D1 (current VDS UUID, uplink-profile UUID, IP-pool UUID, both TZ UUIDs):

```
cat > /tmp/tnp.json <<'EOF'
{
  "display_name": "vcenter-cl01-tnp-01",
  "host_switch_spec": {
    "resource_type": "StandardHostSwitchSpec",
    "host_switches": [{
      "host_switch_name": "vcenter-cl01-vds01",
      "host_switch_id": "50 12 35 19 fa f5 be 0a-71 8d ab e0 b5 e6 b9 cf",
      "host_switch_type": "VDS",
      "host_switch_mode": "STANDARD",
      "host_switch_profile_ids": [{"key": "UplinkHostSwitchProfile", "value": "<UPLINK-PROFILE-UUID>"}],
      "uplinks": [{"vds_uplink_name": "dvUplink1", "uplink_name": "uplink-1"},
{"vds_uplink_name": "dvUplink2", "uplink_name": "uplink-2"}],
      "ip_assignment_spec": {"resource_type": "StaticIpPoolSpec", "ip_pool_id": "<IP-POOL-MANAGER-UUID>"},
      "transport_zone_endpoints": [{"transport_zone_id": "<OVERLAY-TZ-UUID>"}, {"transport_zone_id": "<VLAN-
TZ-UUID>"}]
    }]
  }
}
EOF
curl -sk -u "$P" -X POST "$NSX/api/v1/transport-node-profiles" -H "Content-Type: application/json" -d
@/tmp/tnp.json -w "\nHTTP %{http_code}\n"
```

Expect **HTTP 201**; record the returned TNP `id`.

D3 — Bind the TNP to the cluster (TransportNodeCollection)

```
curl -sk -u "$P" "$NSX/api/v1/fabric/compute-collections" | python -m json.tool | grep -E
'"display_name"|"external_id"'
```

Take the `vcenter-cl01` `external_id` (`<CM-UUID>:domain-cl0`), then POST the TNC (fill the collection id + the TNP id from D2):

```
cat > /tmp/tnc.json <<'EOF'
{
  "display_name": "vcenter-cl01-tnc",
  "resource_type": "TransportNodeCollection",
  "compute_collection_id": "<CM-UUID>:domain-cl0",
  "transport_node_profile_id": "<TNP-UUID-from-D2>"
}
EOF
curl -sk -u "$P" -X POST "$NSX/api/v1/transport-node-collections" -H "Content-Type: application/json" -d
@/tmp/tnc.json -w "\nHTTP %{http_code}\n"
```

This triggers host prep. Per your prior rebuild (schema-doc L-8), **hosts that still have the NSX VIBs transition Not Configured → success in ~3–5 min** — so do **not** pre-clean the hosts; only fall back to the Phase A2 manual cleanup if a host is still not `success` after ~10 min.

(UI alternative for C2–D3: System → Fabric → Transport Zones / Profiles / Transport Node Profiles → Add, then apply to cluster `vcenter-cl01`.)

6. Phase E — Verify

```
# Run in Git Bash or WSL (NOT Windows cmd).
NSX=192.168.1.70; P='admin:<vault>'

# Cluster — look for "overall_status": "STABLE"
curl -sk -u "$P" "https://$NSX/api/v1/cluster/status"

# Compute manager — look for "registration_status": "REGISTERED" and no errors
curl -sk -u "$P" "https://$NSX/api/v1/fabric/compute-managers/<id>/status"

# All 4 transport nodes — look for "state": "success"
curl -sk -u "$P" "https://$NSX/api/v1/transport-nodes/state"

# Tip: append | python -m json.tool to any line above for readable JSON.
```

On a host: `nsxcli -c "get managers"` → Connected; once a test segment exists, `vmkping -I vmk10 -d -s 1572 <peer-TEP>` → 0% loss.

Acceptance: cluster STABLE · CM Registered (no errors) · 4 TNs `success` · 0 alarms · TEP MTU 1600 passes.

7. Phase F — Re-establish SDDC Manager & VCF Operations

1. **SDDC Manager NSX trust** (*parked item, KB 408913*): re-import the **new** NSX cert chain into SDDC Manager's trust stores (`/etc/vmware/vcf/commonsvcs/trusted_certificates.store` + Java cacerts), restart services. The new appliance reissues the cert, so this is done **once, now**.
 2. **SDDC Manager NSX inventory**: reconcile so `GET /v1/nsxt-clusters` shows the cluster **ACTIVE** again (the June-9 DB repair set `nsxt` → ACTIVE; re-validate post-rebuild).
 3. **VCF Operations NSX adapters** (*see Workbook Vol 1 §14 / KB 421016*): import the new NSX cert into VCF Ops Java truststore (`/usr/java/jre-vmware-17/lib/security/cacerts`), set credentials on **both** NSX adapters (VCF `nsx-vip` + standalone "Aria Admin"), Validate → collecting.
-

8. Rollback / fallback

- **Before Phase D succeeds:** the old NSX Manager VM files are still on esxi01 (not yet deleted) — but reverting to them restores the broken state, so this is only a "stop and reassess" point, not a true rollback.
 - **If the fresh rebuild does not reach §6 acceptance:** open a **Broadcom SR** (attach symptoms 8804/8012/90325 + this runbook). Do not hand-hack Corfu/ownership records.
 - **0 segments** throughout means no overlay workload is ever at risk.
-

9. Captured configuration (fill from §1 before executing)

Object	Captured value
Overlay TZ name	vcenter-cl01-tz-overlay01
VLAN TZ name	vcenter-cl01-tz-vlan01
Uplink profile — teaming policy	LOADBALANCE_SRCID ; active uplink-1, uplink-2; standby none; no MTU (uplink-profile MTU = Error 9531)
Uplink profile — Transport VLAN	0 (untagged — all port groups are VLAN-0)
TEP IP pool — CIDR / range / gateway	192.168.13.0/24 / 192.168.13.101-.120 (20 addrs) / 192.168.13.1
TNP — VDS / uplinks / TZ bindings	VDS vcenter-cl01-vds01 ; uplink-1→dvUplink1, uplink-2→dvUplink2; TZ overlay + vlan
NSX admin / audit / root passwords	[vault]

10. Document Information

Field	Value
Title	NSX Manager Redeploy & Transport-Node Rebuild
Document ID	VC-RUNBOOK-NSX-REDEPLOY-20260610
Version	1.6 (DRAFT — review before executing)
Issued	2026-06-10
Author	Virtual Control LLC
Classification	Internal
Platform	VMware Cloud Foundation 9.0.1 — NSX 9.0.1, vCenter 9.0.1, vSAN OSA, 4 nested hosts
Basis	In-place reconcile proven exhausted (June 9 + June 10 live re-test); fresh redeploy, no backup-restore
Sourcing	Broadcom TechDocs (NSX 9.0 deploy/CM/TZ/uplink/IP-pool/TNP/prepare; KB 312644/322412/316037/408913/421016) + live-verified lab values
Related	NSX9-Transport-Node-Profile-Schema-Reference (verified POST bodies — source of Phase C/D); VCF9-Network-Design-As-Built-2026-06-03 §7 (TEP pool / uplink / VLAN values); VCF9-Host-vCenter-Rebuild-Plan P4.9; NSX-Manager-Cold-Start-Troubleshooting-Report Addendum D (2026-05-06 rebuild context); VCF9-Operations-Campaign §3.5/§4.1; PLAN-NSX-Recovery-Options-and-Clean-Rebuild-2026-06-09

Revision History

Version	Date	Notes
1.0	2026-06-10	Initial DRAFT. Built after the June-10 live re-test confirmed the force-delete could not clear the orphaned realization (in-place exhausted). Captures verified lab values (VDS 50 12 35 19..., TZs, TNP, cluster, hosts) and Broadcom NSX-9 deploy/prep + host-cleanup steps. Pending the two \$0.3 decisions before execution.
1.1	2026-06-10	Corrections during walkthrough: A1 now spells out powering off + renaming the old VM (nsx-manager-OLD-...) so the new appliance can reuse the name/IP/FQDN (hostname-vs-VM-label distinction); A2 maintenance-mode value corrected to ensureObjectAccessibility (exact, case-sensitive — live host rejected ensureAccessibility); copy-clean commands.
1.2	2026-06-10	Made self-contained from verified sources (no guesses). Filled the last [confirm] values from the As-Built design + TNP-Schema docs — Transport VLAN 0 , TEP pool 192.168.13.0/24 / .101-.120 / gw 192.168.13.1 , uplink teaming LOADBALANCE_SRCID , no-MTU. Replaced the Phase C/D UI steps with the verified-working POST bodies (TZ / uplink / IP-pool / TNP / TNC), with hard callouts that all per-rebuild UUIDs (VDS = current 50 12 35 19..., TZ/pool/uplink manager-UUIDs, CM-UUID prefix) must be re-fetched live , not copied from the schema doc. Reframed A2 host-VIB cleanup to FALLBACK-only (per L-8, hosts re-prep with VIBs in 3–5 min). Added source cross-references.
1.3	2026-06-11	Phase B made fill-in-the-blank from the OVF descriptor extracted out of the OVA: form factors (small/medium/large/xlarge), the exact configurable property keys (nsx_hostname , nsx_ip_0 , nsx_netmask_0 , nsx_gateway_0 , nsx_dns1_0 , nsx_domain_0 , nsx_ntp_0 , nsx_passwd_0 , nsx_cli_* , SSH toggles) with the mp* /IPv6 fields flagged <i>leave blank</i> (single-node). Corrected the OVA filename to ...24952111.ova (verified on disk). Unique VM name nsx-manager-new (old vSAN folder can't be renamed). License note: NSX runs in eval — no license needed to rebuild.
1.4	2026-06-11	Phase B restructured into 3 copy-clean blocks (B1 discover portgroup+network · B2 set values · B3 deploy) — single-space = so it copies without the two-column scramble; Get-VMHost now uses the FQDN esxi01.lab.local (the IP form fails — vCenter looks up by inventory name); network mapping set via a foreach loop (no <Name> placeholder); NSX passwords via a \$nsxpw variable.
1.5	2026-06-11	Added a plain-English "New to PowerShell?" walkthrough at the top of Phase B — what each block does (B1 look-up / B2 fill-the-form / B3 deploy), how to copy/paste, the <...> placeholder gotcha, "silence = success," sanity-check first, and what the multi-minute import looks like. Written at a beginner level.
1.6	2026-06-11	Phase B corrected after a live deploy failure. The first execution exposed two pitfalls: Import-VApp on the signed .ova hard-fails on the certificate , and the UI deploy used as a workaround silently dropped the OVF customization — the appliance booted to DHCP 192.168.1.13 / hostname nsx-unified-appliance with the .vmx carrying no OVF env. Restructured Phase B from 3 blocks to 5: B0 extract the OVA + delete .cert → import the unsigned .ovf (defeats the cert hard-fail, preserves vAppConfig); B3 deploys without powering on; B4 is a mandatory verify gate (VAppConfig.Property must show nsx_ip_0 / nsx_hostname) before Start-VM . Form factor set to small (16 GB, RAM headroom). Updated the intro note + beginner "big idea" to match.

End of runbook — DRAFT. Do not execute before the \$0.3 decisions and a change window.